



Kernel Uevent

How Information is Passed from Kernel to User Space

This article explores the kernel uevent framework and looks at how drivers use it to send device-specific information to user space. It also covers how to receive and process kernel uevents in user space.

The Linux kernel version 2.6.10 (and later versions) introduced the uevent notification mechanism for kernel and user-space communication. These user events (uevents) generated from the kernel are used by user-space daemons to either create/ remove device files, to run programs, or load/remove a driver in user land. Inside the kernel, these uevents are linked to the kernel data structure called *kobject*. This data structure's life cycle (linked with a device) is what is notified as uevents to user space.

Netlink

The Linux kernel uses netlink to send kernel uevents to the user space. Netlink is a socket-like mechanism used in Linux to pass information between kernel and user processes. Netlink, similar to a generic BSD socket infrastructure, supports primitive APIs like *socket()*, *bind()*, *sendmsg()* and *recvmsg()*.

Uevent actions

As discussed above, a uevent message communicates the device's or subsystems' states inside the kernel to user space. The *kobject_action* data structure indicates the kernel object's

state, and is defined in *include/linux/kobject.h* as *enum kobject_action*:

```
enum kobject_action {
    KOBJ_ADD,
    KOBJ_REMOVE,
    KOBJ_CHANGE,
    KOBJ_MOVE,
    KOBJ_ONLINE,
    KOBJ_OFFLINE,
    KOBJ_MAX
};
```

Each uevent message sent to user space is tagged with one of these *kobject* actions. For example, actions like *KOBJ_ADD* or *KOBJ_REMOVE* are used to notify the addition or deletion of a kernel object—which happen when a device is added or deleted inside the kernel, using *device_add* or *device_del*. *KOBJ_CHANGE* is the action most used by drivers to notify changes in the device, like the configuration or state. Other actions like *KOBJ_ONLINE*/*KOBJ_OFFLINE* are generally